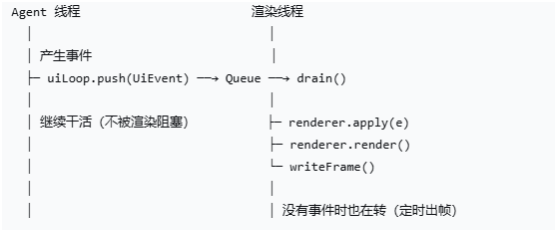
L9-v7

2026年10月24日 16:48

L8 ： UI “在回调里同步 std::cout”——工具跑 1 秒，画面就卡 1 秒。
L9 ：把渲染从 Agent 线程里剥出来，做成独立线程 + 事件队列。
回调只做“入队 + 立刻返回”，渲染线程自己去出帧——**异步**



1. Agent 线程从不直接输出——它只 push 事件，然后立刻返回。push 是"入队 + 通知"，不涉及 I/O。
2. 渲染线程定时出帧——它不看"有没有事件"，看"到没到出帧时间"。所以工具执行 30 秒，它出 30 秒的帧，spinner 一直在转。
3. 权限弹窗阻塞 Agent 线程，不阻塞渲染线程——askPermission 在 Agent 线程上等 std::getline，渲染线程照常出帧。

新增清单

文件	变化类型	核心新增
ui/tui.hpp	新增文件	UiEvent / EventQueue / IRenderer / UiLoop / stripAnsi / makeRenderer / stdoutIsTty
main.cpp	重大修改	① --ui-demo；② toUiEvent() 翻译层；③ 观察者接线改成 push 事件；④ hooks.confirm 真接线；⑤ AF_UI 环境变量；⑥ I-39 ~ I-49 自检；⑦ 删掉旧的 struct Renderer

渲染层的事件类型

```
struct UiEvent {
    enum class Kind {
        Text, Thinking, ToolCall, ToolArgs, ToolStart, ToolDone,
        Usage, Notice, Error, StreamError, StreamDone, PermAsk
    };
    Kind kind = Kind::Text;
    std::string text;
    std::string toolName, toolId;
    double elapsedMs = 0;
    int tokens = 0;
    bool isError = false;
    int promptId = 0;
};
```

EventQueue —— 有界 FIFO

```
class EventQueue {
public:
    explicit EventQueue(size_t cap);
    bool push(UiEvent e);        // 满了返回 false（背压）
    std::vector<UiEvent> drain(); // 取空
    size_t size() const;
};
```

操作	调用者	语义
push	Agent 线程	满则阻塞；closed 则直接返回
drain	渲染线程	一次取空，通知生产者"有空位了"
close	收尾	唤醒所有等待者，push 不再阻塞

```
void EventQueue::push(UiEvent e) {
    std::unique_lock<std::mutex> lk(mu_);
    notFull_.wait(lk, [&] { return q_.size() < cap_ || closed_; });
    if (closed_) return;
    q_.push_back(std::move(e));
    notEmpty_.notify_one();
}
```

IRenderer —— 两个后端

```
class IRenderer {
public:
    virtual ~IRenderer() = default;
    virtual const char* name() const = 0;
    virtual void apply(const UiEvent& e) = 0;   // 吸收事件
    virtual std::string render() = 0;           // 出一帧
    virtual bool toolInFlight() const = 0;      // 有没有工具在跑
    virtual bool askPermission(const std::string& prompt, int id) = 0;
};

std::unique_ptr<IRenderer> makeRenderer(bool wantTui);
```

两个实现：

实现	模式	输出	用途
FtxuiRenderer	画布（可重画）	ANSI + Element 树	TTY 时用，spinner、耗时条
PlainRenderer	行打印机（不可重画）	纯文本，零 ANSI	非 TTY，可 grep

UiLoop —— 渲染线程

```
class UiLoop {
public:
    UiLoop(std::unique_ptr<Renderer> r, bool isTty);
    ~UiLoop();

    UiLoop(const UiLoop&)            = delete;
    UiLoop& operator=(const UiLoop&) = delete;

    void push(UiEvent e);              // ★ 非阻塞语义：只做入队，立刻返回
    void stop();                       // 关闭队列 + join（幂等）
    Renderer& renderer() { return *r_; }

    int events() const { return events_.load(); }   // 入队事件数（C28）
    int frames() const { return frames_.load(); }   // 出帧数（C29 / C30）
    int  spinnerTicks() const { return spins_.load(); }

private:
    void loop();

    std::unique_ptr<Renderer> r_;
    bool                      isTty_;
    EventQueue                q_;
    std::thread               th_;
    std::atomic<int>          events_{0}, frames_{0}, spins_{0};
    std::atomic<bool>         running_{true};
};
```

核心机制：Agent 线程 push 事件，渲染线程在自己的循环里 drain + render。

```
std::string render() override {
    Elements body;
    body.push_back(text("AgentForge v7 (画布模式) ") | bold);

    // 对话流（只显示最后 14 行）
    for (std::size_t i = from; i < log_.size(); ++i)
        body.push_back(text(log_[i]));

    // 工具卡片
    for (const auto& t : tools_) {
        if (t.done) {
            os << (t.err ? " x " : " v ") << t.name << "  " << t.ms << "ms";
        } else {
            const double ms = duration(Clock::now() - t.startedAt).count();
            os << " " << kSpin[spinIdx_] << " " << t.name << "  执行中 " << ms << "ms";
        }
    }

    // 权限弹窗（overlay）
    if (pending_) {
        body.push_back(text("[权限] " + pendingPrompt_) | border);
    }

    // 离屏渲染：Element 树 → 字符矩阵 → 字符串
    auto screen = Screen::Create(Full(), Fit(doc));
    Render(screen, doc);
    return screen.ToString();
}
```

```
void UiLoop::loop() {
    bool dirty = false;
    auto lastSpin = Clock::now();

    // ★ 退出条件是 "running 假 且 队列空"
    while (running_.load() || q_.size() > 0) {
        const auto frameStart = Clock::now();

        // ① 消费所有待处理事件
        for (const auto& e : q_.drain()) { r_->apply(e); dirty = true; }

        // ② 工具执行期间，即使没有事件也出帧
        if (r_->toolInFlight() && frameStart - lastSpin >= 300ms) {
            lastSpin = frameStart;
            spins_.fetch_add(1);
            dirty = true;
        }

        // ③ 有变化就画一帧
        if (dirty) {
            writeFrame(r_->render(), isTty_);
            frames_.fetch_add(1);
            dirty = false;
        }

        // ④ 限速 ~60fps
        std::this_thread::sleep_until(frameStart + 16ms);
    }

    // 收尾：把剩下的事件全部消费掉再走
    for (const auto& e : q_.drain()) r_->apply(e);
    writeFrame(r_->render(), isTty_);
}
```

```
struct LoopHooks {
    std::function<void(const ChatChunk&)>    onChunk;      // 流式增量
    std::function<void(const ToolCall&)>     onToolStart;
    std::function<void(const ToolResult&)>   onToolDone;
    std::function<void(const std::string&)>  onNotice;
    std::function<bool(const std::string&)>  confirm;      // 权限询问
    std::function<void(const std::string&)>  onError;
};
```

```
hooks.confirm = [&](const std::string& prompt) -> bool {
    return uiLoop.renderer().askPermission(prompt, ++permId);
};
```

```
hooks.onChunk = [&](const ChatChunk& c) {
    ++chunks;
    if (std::holds_alternative<TextDelta>(c) &&
        !std::get<TextDelta>(c).text.empty()) sawFirstToken = true;
    uiLoop.push(toUiEvent(c));          // ★ 立刻返回
};
```

```
static af::ui::UiEvent toUiEvent(const ChatChunk& c) {
    UiEvent e;
    std::visit(overloaded{
        [&](const TextDelta& d) { e.kind = UiEvent::Kind::Text; e.text = d.text; },
        [&](const ToolCallStart& s) {
            e.kind = UiEvent::Kind::ToolCall; e.toolName = s.name; e.toolId = s.id;
        },
        [&](const ToolCallDelta&) {
            // 半个 JSON：不落地。给个空 Text，两个渲染器都会忽略它。
            e.kind = UiEvent::Kind::Text;
        },
        // ...
    }, c);
    return e;
}
```

```
UiLoop::UiLoop(std::unique_ptr<Renderer> r, bool isTty)
    : r_(std::move(r)), isTty_(isTty) {
    th_ = std::thread([this] { loop(); });
}
```

完整数据流

```
时间轴                Agent 线程                    渲染线程
  │                     │                            │
  t=0                   │ hooks.onToolStart(Read)    │
  │                     ├─ toolStartedAt["t1"]=now   │
  │                     └─ push(ToolStart) ──→ Queue ──→ drain → apply
  │                     │                            ├─ inFlight_ = true
  │                     │                            └─ render() → spinner
  │                     │                            │
  t=0.3                 │ (阻塞在 registry_.dispatch)  ├─ 无新事件，但 toolInFlight
  │                     │                            │   → 强制出帧 → spinner 转
  │                     │                            │
  t=0.6                 │ (仍在执行)                   ├─ 同上
  │                     │                            │
  t=1.2                 │ registry_.dispatch 返回      │
  │                     │ hooks.onToolDone(...)      │
  │                     ├─ 算 ms = 1200              │
  │                     └─ push(ToolDone) ──→ Queue ──→ drain → apply
  │                     │                            ├─ inFlight_ = false
  │                     │                            └─ render() → "v Read 1200ms"
  │                     │                            │
```

```
Agent 线程                          渲染线程
  │                                   │
  │ hooks.confirm(prompt)             │
  │   └─ renderer.askPermission()     │
  │        └─ push(PermAsk) ──→ Queue ──→ drain → apply
  │        └─ getline(std::cin)       ├─ pending_ = true
  │           (阻塞，等用户输入)       ├─ render() → 弹窗出现
  │                                   ├─ 继续出帧（弹窗不阻塞渲染）
  │                                   │
  │ 用户输入 'y'                      │
  │        └─ return true             │
  │                                   ├─ push(PermAnswer) → drain
  │                                   └─ pending_ = false
```